

Orbital Academy

Documentação base da solução · v1.1

Campo	Definição
Projeto	Global Solution FIAP — Space Connect — 2026.1
Curso	Engenharia de Software, 3º semestre
Tema	Tecnologia espacial aplicada a desafios reais
Solução	Orbital Academy
Aba de conteúdo	Espaçoteca
Componente operacional central	Console de Missão, com Motor de Decisão (ML + Otimização)
Missão-bandeira do MVP	Risco em lavoura (agro), com dado real de NASA/INPE
Disciplinas (8)	IA & ML; Physical Computing IoT/IoB; Mobile Development & IoT; Operating Systems; Testing, Compliance & QA; C# Software Development; Arquitetura Orientada a Serviços (SOA) & Web Services; Cybersecurity
Marcos conhecidos	Mobile, C# e SOA: entrega em 09/06/2026 (Mobile às 23h55). Demais datas: a confirmar com os professores
Congelamento ideal antes do pitch	Travar a versão final com pelo menos 1 semana de folga para ensaios e contingência
Versão	1.1 — incorpora as 8 matérias (inclui C#, SOA e Cybersecurity); camada de serviço .NET; seção de segurança dedicada

Tese central

O Orbital Academy ensina espaço operando. O usuário pega dado real de satélite, o modelo prevê o risco, ele decide onde usar o recurso escasso, o otimizador acha a melhor alocação e ele vê o impacto. Educação na alma, motor de verdade embaixo. Quando dado espacial vira decisão real, qualquer pessoa pode ser operadora.

Operar é aprender. Decidir é o impacto.

Sumário

1. Resumo executivo
2. Contexto e problema
3. Tese da solução
4. Componentes da solução
5. Console de Missão e Motor de Decisão
6. Jornada do operador

7. Arquitetura técnica
8. Dados, inteligência e otimização
9. Aplicativo (mobile e versão web)
10. Backend, serviços e banco
11. Cibersegurança
12. Qualidade e conformidade
13. Métricas de sucesso
14. Escopo do protótipo mínimo
15. Plano de execução
16. Divisão de trabalho
17. Roteiro de demonstração
18. Perguntas difíceis da banca
19. Fechamento

1. Resumo executivo

O Orbital Academy é uma plataforma que ensina qualquer pessoa a transformar dado espacial em decisão real, operando uma missão de verdade em vez de assistir a uma aula. A solução combina dado aberto de satélite (NASA Earthdata, NASA FIRMS, INPE, Copernicus), um modelo de machine learning tradicional que prevê risco e um motor de otimização próprio que decide como alocar recursos limitados. O Console de Missão é a interface onde a pessoa vive esse ciclo. A Espaçoteca é a aba complementar de conteúdo e exploração.

A solução não é um app educacional do tipo museu/quiz e não é apenas um dashboard de monitoramento. Ela resolve uma falha estrutural: dado espacial é abundante e aberto, mas decidir o que fazer com ele continua sendo uma capacidade de especialista. O Orbital Academy democratiza essa capacidade através de uma experiência de operação.

Componentes principais

Parte	Papel na solução
Console de Missão	Interface onde o usuário vê dado, decide, roda o motor e mede impacto.
Motor de Decisão (camada 1)	Modelo de ML tradicional que prevê risco/perda a partir de dado estruturado, com métrica e interpretação.
Motor de Decisão (camada 2)	Otimizador próprio que aloca recursos limitados sobre a previsão (a parte 'algo próprio').
Câmera de Validação	Visão computacional (OpenCV/MediaPipe) na webcam que confirma ou corrige o que o satélite previu.
Serviço .NET de Catálogo	API em C#/.NET que modela Satélites, Sensores e Alertas e os expõe via web services. Cobre C# e SOA.
App de Campo	React Native + Expo, com AsyncStorage. Funciona offline; o build web atende os professores.
Backend com Continuidade Espacial	Windows Server (AD/DNS/IIS) hospedando portal e API, com DR replicando para datacenter espacial.

Parte	Papel na solução
Camada de Cibersegurança	Controles de segurança (IAM, criptografia, hardening, logs) protegendo a integridade da decisão.
Espaçoteca	Aba secundária com conteúdo educativo, glossário, missões históricas e exploração de APIs.

Frase de posicionamento

Operar é aprender. Decidir é o impacto.

2. Contexto e problema

O tema da Global Solution 2026.1 da FIAP é tecnologia espacial aplicada a desafios reais. A live de abertura, com a Bizu Space, a Safe on Orbit e a Visiona, deixou três pontos consistentes: o mercado real está em downstream (aplicação dos dados), o dado relevante já é aberto (Landsat, Sentinel, INPE, NASA Earthdata), e a entrega que importa é virar dado em decisão. O Guilherme da Safe on Orbit foi específico ao dizer que o que eles fazem internamente não é IA generativa, é otimização clássica sobre cenários com variáveis aleatórias e recursos limitados.

2.1 Problema prático

Dado espacial é abundante, é gratuito e é cada vez mais fácil de consumir. Mas saber o que fazer com ele continua sendo uma capacidade restrita a quem tem formação técnica específica. Para o pequeno produtor rural, para a equipe de Defesa Civil de um município pequeno, para um agente de saúde em campo, o satélite ainda parece distante. A pergunta não é mais 'como vemos?'. A pergunta é 'quem decide?'.

Em paralelo, FIAP e o ecossistema brasileiro de tecnologia carregam a 'síndrome do vira-lata' que o Arthur da Visiona citou na live: a ideia de que espaço é coisa de gênio da NASA. Isso afasta jovens da área e perpetua a dependência de capacidades estrangeiras justamente em um momento em que o downstream está crescendo.

O problema real pode ser formulado assim:

Como tornar a decisão baseada em dado espacial operável por qualquer pessoa, sem perder o rigor técnico?

2.2 Por que descartamos três caminhos comuns

Caminho descartado	Por que não
App educacional do tipo museu/quiz	Mostra dado, não muda decisão. Não usa as exigências técnicas da GS de forma natural e força encaixe artificial das matérias.
Mais um app de alerta de desastre	É o gênero da edição passada da GS (eventos extremos). Risco alto de a banca pattern-matching para soluções já vistas. Sem diferencial, perde por familiaridade.
Sistema que tenta detectar risco melhor que a NASA	A NASA (FIRMS) e o INPE já fazem detecção em tempo quase real, de graça. O que está aberto é decidir o que fazer com a detecção.

3. Tese da solução

Decisão com dado espacial não acontece quando alguém vê um mapa colorido. Acontece quando alguém escolhe, sob recurso limitado, qual ação tomar primeiro, e mede o resultado. O Orbital Academy organiza essa transformação em sete movimentos:

1. Ver: o satélite acende uma área de risco no mapa, com dado aberto e real.
2. Prever: o modelo de ML estima a probabilidade de perda e o motivo principal.
3. Validar: a câmera na ponta confirma ou corrige a previsão do satélite, em campo.
4. Decidir: o operador escolhe onde alocar o recurso escasso (água, equipe, tempo).
5. Otimizar: o motor próprio recalcula a melhor alocação dada as restrições atuais.
6. Agir: o app guia a execução, inclusive offline.
7. Medir: o sistema registra impacto e realimenta o modelo e a próxima decisão.

Esse ciclo é genérico. A missão-bandeira do MVP é agro (risco em lavoura), porque é o domínio onde a democratização tem mais impacto para o pequeno produtor brasileiro e foi a proposta inicial do grupo. Outras missões (queimada, risco hídrico, deslizamento) usam o mesmo motor e entram no roadmap.

4. Componentes da solução

4.1 Console de Missão

Interface principal por onde a pessoa opera uma missão. Mostra o mapa de risco, o ranking das áreas priorizadas, o motivo principal de cada previsão, a fila de decisão e o impacto consolidado. Existe em duas faces: web (acessada pelo navegador, inclusive pelos professores) e mobile (app de campo).

4.2 Motor de Decisão

Cérebro técnico, em duas camadas. A camada 1 é um modelo de ML tradicional (Random Forest e Regressão Logística comparados) que prevê risco/perda por área, com métrica clara (F1, AUC) e interpretação (importâncias). A camada 2 é um otimizador próprio (busca local, algoritmo genético ou programação linear via PuLP) que aloca recursos limitados sobre a previsão. Próprio significa que a formulação do problema e a implementação são do time.

4.3 Câmera de Validação

Componente de visão computacional. O operador aponta a webcam (no notebook em demo, no celular em campo) para uma folha de cultura ou amostra, e o sistema classifica o estado (saudável, estresse hídrico, praga, doença). Essa resposta valida ou corrige a previsão do satélite e fecha o loop de ground-truth.

4.4 App de Campo

Versão mobile do Console de Missão, em React Native com Expo. Como o Expo gera build web, o mesmo código atende a aba para professores. Funciona offline com AsyncStorage e sincroniza ao voltar a conectividade. O cenário rural (sítio sem sinal) torna o offline narrativamente justificado.

4.5 Backend com Continuidade Espacial

Servidor Windows com Active Directory, DNS e IIS hospedando o portal e a API. Inclui plano de Disaster Recovery que replica o plano de controle para um datacenter representado como espacial, mantendo a operação viva quando a infra terrestre cai.

4.6 Espaçoteca

Aba complementar dentro do Orbital Academy, focada em conteúdo: glossário, missões históricas em modo treino, exploração das APIs de NASA e INPE. A Espaçoteca não é o coração do produto; ela apoia o aprendizado do operador, que continua sendo a missão real. Operar continua sendo a forma principal de aprender.

4.7 Serviço .NET de Catálogo (C# + SOA)

Serviço em C#/.NET que modela o domínio espacial em POO: Satélite, Sensor e Alerta, com herança e polimorfismo (cada tipo de sensor lê de um jeito; cada tipo de alerta dispara de um jeito), interfaces e injeção de dependência, manipulação de DateTime para histórico, tratamento de exceções específico (sistemas críticos espaciais não podem quebrar abruptamente) e VO/DTO para organização. Ele conecta com banco e se expõe via web services, consumido pelo backend Python e pelo app. Um único artefato cobre duas disciplinas: C# Software Development (qualidade de POO em .NET) e SOA & Web Services (serviços integrados + banco). Como SOA aceita qualquer linguagem e dispensa aplicação distinta, esse desenho satisfaz os dois professores sem trabalho duplicado.

5. Console de Missão e Motor de Decisão

5.1 Justificativa do design

A premissa é que qualquer pessoa, inclusive um jurado da banca, consegue sentar no Console e operar uma missão até o fim em poucos minutos. Isso é demonstrado ao vivo, não declarado. A interface evita jargão e enquadra cada decisão como uma pergunta clara: o que fazer primeiro, onde colocar o recurso, qual é o resultado.

5.2 Princípios

Princípio	Como aparece no produto
Operar é aprender	O usuário não tem 'modo aula' separado; ele aprende sob pressão de uma decisão real.
Transparência do motor	O modelo mostra o motivo principal de cada previsão; o otimizador mostra por que escolheu cada alocação.
Ground-truth obrigatório	A câmera valida o que o satélite disse. Previsão sem validação é tratada como provisória.
Recurso é finito	Cada missão tem orçamento, equipe e prazo; nada é ilimitado.
Continuidade	Se a rede cai, o app segue offline; se o datacenter cai, o DR mantém o controle vivo.
Auditoria	Toda decisão e seu resultado ficam registrados, para realimentar o motor.

5.3 Estados de uma missão

Estado	Significado
Identificada	Satélite e modelo apontaram a área; ainda não há decisão humana.

Estado	Significado
Validada	Câmera confirmou ou corrigiu a previsão; missão pronta para decisão.
Decidida	Operador escolheu a ação; otimizador propôs a alocação.
Em execução	Equipe está aplicando a ação; app de campo registra progresso.
Concluída	Ação finalizada; impacto medido.
Reprogramada	Sem resultado claro; volta para a fila com novos dados.
Perdida	Não foi possível agir a tempo; serve de aprendizado.

6. Jornada do operador

6.1 Antes da missão

8. Satélite e modelo acendem áreas de risco no mapa.
9. Ranking organiza as áreas pelo score, mostrando o motivo de cada uma.
10. Operador seleciona as missões do dia, dentro da capacidade da equipe.
11. Cada missão recebe responsável, prazo e recurso alocado.

6.2 Durante a missão

12. Operador em campo abre o app e vê a missão atribuída.
13. Aponta a câmera na cultura para validar a previsão.
14. App mostra a ação recomendada e o recurso disponível.
15. Operador executa, registra resultado e move a missão de estado.
16. Se não há rede, o app continua funcionando e enfileira a sincronização.

6.3 Depois da missão

17. Resultado é sincronizado quando a rede volta.
18. Indicadores atualizam: % salva, perda evitada em R\$, tempo até decisão.
19. Otimizador é informado e recalcula a fila para a próxima rodada.
20. Casos não respondidos voltam para reprogramação.

7. Arquitetura técnica

A arquitetura deve ser simples, modular e demonstrável dentro do semestre. O objetivo é cumprir os critérios técnicos das oito disciplinas sem inflar o escopo, com cada matéria mapeada a um artefato real.

7.1 Mapa das 8 matérias e seus artefatos

Matéria	Artefato no projeto	Frente / responsável
IA & ML	Notebook .ipynb com modelo de risco (RF/Logística), métrica e interpretação	A confirmar (proposta: Abner)

Matéria	Artefato no projeto	Frente / responsável
Physical Computing IoT & IoB	Módulo de webcam (OpenCV/MediaPipe) integrado ao app	A confirmar (proposta: Beatriz)
Mobile Development & IoT	App React Native + Expo (build mobile + web)	Mariana
C# Software Development	Serviço .NET de Catálogo (Satélite/Sensor/Alerta) com POO, DI, exceções	A confirmar (proposta: Eduardo)
SOA & Web Services	Mesmo serviço .NET + API Python integrados via web services + banco	A confirmar (proposta: Eduardo)
Operating Systems	Windows Server (AD/DNS/IIS) hospedando o .NET + plano de DR espacial	A confirmar (proposta: Eduardo)
Cybersecurity	Doc técnico ≤6p + repo com hands-on (recomendado: IAM/JWT)	Heloísa
Testing, Compliance & QA	Arquitetura TOGAF no Archi + doc de impacto + vídeo pitch	Heloísa

7.2 Camadas técnicas

Camada	Tecnologias sugeridas	Função
Dados	SQLite no protótipo; PostgreSQL/SQL Server se houver folga	Persistir áreas, missões, ações, resultados, catálogo e snapshots.
IA / ML	Python, Jupyter, pandas, scikit-learn	Treinar e avaliar o modelo de previsão; gerar interpretação.
Otimização	Python, PuLP ou implementação própria (busca local, GA)	Alocar recurso limitado sobre a previsão.
Visão	Python, OpenCV, MediaPipe	Classificar imagens da câmera para validar a previsão.
API principal	Python, FastAPI, Pydantic, Uvicorn	Endpoints REST para Console, app e otimizador.
Serviço de Catálogo	C#, .NET Core API, EF Core, banco relacional	Satélites/Sensores/Alertas via web services. Cobre C# e SOA.
Mobile / Web	React Native, Expo, AsyncStorage, TypeScript	App offline-first; build web atende o site dos professores.
Infra / SO	Windows Server, AD, DNS, IIS; DR para datacenter espacial	Hospedar portal, API e o serviço .NET; continuidade.
Segurança	JWT/IAM, TLS, hardening GPO, logs, backup	Proteger integridade da decisão e dos dados.

Camada	Tecnologias sugeridas	Função
Arquitetura	Archi (ArchiMate / TOGAF)	Modelar negócio, sistemas e tecnologia.

7.3 Visão de fluxo técnico

21. Datasets abertos (NASA Earthdata, FIRMS, INPE, Copernicus, INMET) alimentam o notebook de ML.
22. Notebook treina o modelo e exporta o resultado (arquivo ou serviço).
23. O serviço .NET de Catálogo guarda e expõe metadados de satélites, sensores e alertas via web services.
24. API principal (Python) consulta modelo e catálogo e expõe o ranking de áreas.
25. Console e app consomem a API e mostram a missão.
26. Câmera roda inferência local; resultado vai para a API.
27. Operador decide; o otimizador roda e devolve a alocação.
28. Ação e resultado são registrados; indicadores atualizam.
29. Se o datacenter terrestre cai, o DR no datacenter espacial mantém API e painel disponíveis.

8. Dados, inteligência e otimização

8.1 Objetivo das camadas

A camada 1 prevê risco de perda por área e explica o motivo principal. A camada 2 transforma essa previsão em ação concreta, dada uma restrição de recurso. Sem a camada 1 não há previsão; sem a camada 2 a previsão vira só mais um dashboard.

8.2 Fontes de dado

Fonte	Conteúdo	Uso
NASA Earthdata / GIBS	NDVI, temperatura de superfície	Sinal de saúde da vegetação e risco hídrico.
NASA FIRMS	Fogo ativo (MODIS, VIIRS)	Missão de incêndio (roadmap) e contexto climático.
INPE TerraBrasilis / Queimadas	Uso da terra e queimadas no Brasil	Calibração nacional.
Copernicus / Sentinel	Imagens de alta resolução	Validação cruzada.
INMET	Dados meteorológicos	Chuva, temperatura, umidade.
EMBRAPA (open data)	Dados agronômicos	Parâmetros para a missão agro.

8.3 Métricas do modelo (camada 1)

- Métrica primária: F1 ponderada.
- Métrica secundária: AUC ROC.
- Comparação obrigatória: Regressão Logística (baseline) vs Random Forest.

- Interpretação: top-3 importâncias por classe e por área.
- Cuidado anti-vazamento: features futuras não entram para prever o passado.

8.4 Formulação da otimização (camada 2)

- Áreas com score de risco e impacto potencial.
- Recursos com capacidade e custo.
- Restrições: capacidade total, janela de tempo, prioridades.
- Objetivo: maximizar impacto esperado salvo, sujeito às restrições.
- Saída: alocação por área, ordem de execução, recurso comprometido.

Implementação inicial: PuLP sobre instância pequena. Alternativa: busca local/algoritmo genético implementado pelo time. Em qualquer caso, comparar com uma heurística greedy para mostrar o ganho do otimizador.

9. Aplicativo (mobile e versão web)

O app é entrega da disciplina de Mobile. Por ser React Native com Expo, o mesmo código gera build web, que atende a aba acessada pelos professores. Não há duas frentes de front-end.

Tela	Função técnica
Entrar	Autenticação do operador (token JWT).
Mapa de risco	Áreas coloridas por score; toque abre a área.
Detalhe da área	Perfil, score, motivo principal, ações recomendadas.
Câmera de validação	Câmera abre e classifica a amostra em tempo real.
Decisão e otimização	Operador define recursos; sistema mostra a alocação ótima.
Fila offline	Ações aguardando sincronização; status visível.
Indicadores	Impacto consolidado: salvo, perdido, em execução, R\$.

Pré-requisitos da disciplina já atendidos: cinco ou mais telas navegáveis, persistência local com AsyncStorage, estilização customizada, organização de arquivos e README com login de teste.

10. Backend, serviços e banco

10.1 Entidades principais

Entidade	Campos principais
Usuario	id, nome, papel (operador/líder/admin), unidade
Area	id, lat, lng, cultura, tamanho, dono
Satelite / Sensor / Alerta	catálogo modelado no serviço .NET (POO, herança, polimorfismo)
Observacao	id, area_id, fonte (satélite/câmera/sensor), tipo, valor, data
Previsao	id, area_id, score, classe, motivo_principal, modelo_versao, data

Entidade	Campos principais
Recurso	id, tipo (água/insumo/equipe/veículo), capacidade, custo
Missao	id, area_id, prioridade, status, responsavel_id, prazo
Acao	id, missao_id, tipo, recurso_alocado, data
Resultado	missao_id, desfecho, impacto, perda_evitada_rs

10.2 Endpoints mínimos

Endpoint	Método	Função
/areas	GET	Listar áreas.
/risco/ranking	GET	Áreas priorizadas pelo modelo.
/catalogo/satelites	GET	Servido pelo serviço .NET (web service).
/missoes	POST/GET	Criar e listar missões.
/missoes/{id}/status	PATCH	Atualizar estado.
/validar	POST	Receber inferência da câmera.
/otimizar	POST	Rodar o motor de otimização.
/indicadores	GET	Métricas agregadas.

10.3 Serviço .NET de Catálogo (C# + SOA)

O catálogo de Satélites, Sensores e Alertas vive em um serviço .NET separado, consumido pela API principal via web services. É onde mora a modelagem de domínio em POO exigida por C# e a integração de serviços + banco exigida por SOA. A API Python orquestra: pede ao serviço .NET os metadados de origem do dado, roda o modelo e o otimizador, e devolve a decisão. Essa separação é a espinha SOA do projeto.

10.4 Continuidade espacial (DR)

O plano de DR replica banco e serviços para um datacenter representado como espacial. Em catástrofe regional, o app aponta para a réplica, que responde pelas mesmas rotas. A demonstração simula a queda do datacenter principal e mostra o app seguindo contra a réplica.

11. Cibersegurança

Módulo de cibersegurança obrigatório, estruturado conforme o brief: identificação de riscos, controles, implementação prática e conexão com a GS e os ODS. A integridade da decisão é central: sem segurança, 'democratizar a decisão espacial' vira 'democratizar a decisão envenenada'. Esta seção alimenta diretamente o documento técnico (máx. 6 páginas) e o repositório da disciplina.

11.1 Ativos críticos

Ativo	Por que é crítico
API principal (Python) + serviço .NET	Ponto de acesso central; comprometer derruba todo o sistema.
Banco de dados	Histórico de decisões e dados de produtor (PII). Vazar = risco LGPD; apagar = perda de rastreabilidade.
Pipeline de ingestão de dado satelital	Manipular o dado de origem (NASA/INPE) envenena a previsão.
Canal de DR para o datacenter espacial	A continuidade da decisão depende deste link; interceptar = decisão errada em catástrofe.
Console de Missão (painel)	Superfície administrativa; acesso indevido gera decisões falsas em nome do operador.

11.2 Identificação de ameaças (STRIDE simplificado)

Escolhemos STRIDE por cobrir a taxonomia completa de forma didática. Mapeamento básico:

Categoria	Ameaça no Orbital Academy	Impacto
Spoofing	Atacante se passa por operador autorizado	Decisões e alocações maliciosas
Tampering	Manipulação de dado satelital antes do modelo	Previsão envenenada, ação prejudicial em campo
Repudiation	Operador nega ação registrada	Perda de rastreabilidade e auditoria
Information Disclosure	Vazamento de dados de produtor (PII)	Risco LGPD, perda de confiança
Denial of Service	DDoS na API durante uma missão	Decisão paralisada no momento crítico
Elevation of Privilege	Operador comum acessa funções de admin	Configurações alteradas, escalada de impacto

11.3 Controles de segurança (mínimo 4)

Controle	Aplicação no projeto
IAM com papéis	Operador, líder e admin com permissões distintas; token JWT com claim de papel. Pode usar o AD do Windows Server como fonte.
Criptografia (trânsito e repouso)	HTTPS/TLS em toda a API; campos sensíveis (PII de produtor) criptografados no banco.
Hardening do Windows Server	GPO restritivas, portas não usadas fechadas, atualizações automáticas, logs habilitados.
Monitoramento de logs	Captura de autenticação, erros 5xx, picos de tráfego e atualização de modelo; alerta de anomalia.

Controle	Aplicação no projeto
Backup e recuperação	Backup diário do banco; DR para datacenter espacial reconhecido também como controle de segurança.

O diagrama simples mostrando onde cada controle se aplica será feito pela frente de QA/Cyber (Archi ou Draw.io), reaproveitando a arquitetura TOGAF.

11.4 Implementação prática (hands-on)

Recomendação: implementar IAM mínima com JWT e papéis na API, demonstrando acesso permitido vs negado por papel. Evidências: trechos de código, prints dos testes e log de autenticação. Razão: integra com o que a API precisa de qualquer forma, gera evidência limpa e conversa com Operating Systems (o AD como fonte dos papéis).

Plano B mais leve: criptografar um campo de PII (nome/documento do produtor) no banco, mostrando o valor antes e depois. Em qualquer caso, a implementação é pequena mas real, com evidências.

11.5 Conexão com a GS e os ODS

A segurança protege a tese central: a decisão espacial precisa chegar íntegra na ponta. Conexões com os ODS: ODS 9 (infraestrutura e inovação robustas), ODS 16 (instituições eficazes, via rastreabilidade e auditoria), ODS 2 (proteger decisões agrícolas de manipulação) e ODS 11 (cidades sustentáveis, quando o motor for aplicado a desastres urbanos no roadmap).

12. Qualidade e conformidade

12.1 Critérios de qualidade

Critério	Como demonstrar
Usabilidade	Pessoa leiga (inclusive o jurado) opera a missão até o fim em poucos minutos, ao vivo.
Confiabilidade	Missões e ações persistem; estados não se perdem; fila offline sincroniza sem duplicar.
Desempenho	Ranking e otimização respondem em segundos na instância de demo.
Explicabilidade	Cada previsão mostra motivo principal; cada alocação mostra critério.
Rastreabilidade	Toda missão tem trilha de ações e resultado.
Aderência ao negócio	Indicadores conectam decisão a impacto (R\$, % salvo).

12.2 Riscos e mitigação

Risco	Mitigação
Parecer 'mais um app de NDVI'	Herói é o motor de decisão, não o mapa. Demo lidera pela decisão e pela otimização.
Satélite parecer decorativo	Dado aberto real alimenta o modelo; sem o dado, não há previsão.

Risco	Mitigação
Modelo parecer caixa-preta	Interpretação, comparação de dois modelos, métrica exposta.
DR espacial parecer forçado	Enquadrar como continuidade quando a infra cai; demonstrar o failover.
AsyncStorage parecer enfeite	Cenário rural sem sinal; demo corta a conexão ao vivo.
Escopo inflar com 'todas as missões'	Uma missão (agro) profunda no MVP; outras como roadmap.
8 matérias virarem 8 trabalhos colados	Arquitetura única: um artefato por matéria, todos integrados via a API e o serviço .NET.

12.3 Entradas para a arquitetura TOGAF (Heloísa)

- Partes interessadas: operador, líder, agente em campo, pequeno produtor, professor/jurado, time, parceiros de dado aberto.
- Arquitetura de negócio: missão como unidade de valor; serviços de previsão, validação, decisão, execução e medição.
- Arquitetura de sistemas: Console, app, API Python, serviço .NET, modelo, otimizador, integração de dados.
- Arquitetura de tecnologia: Windows Server, FastAPI, .NET, banco, DR espacial, camada de segurança.
- Impacto: perda evitada em R\$, produtores potencialmente atendidos, redução da dependência de especialista.
- Dados públicos para o doc de impacto: INPE, EMBRAPA, IBGE, CONAB.

13. Métricas de sucesso

Categoria	Métrica
Modelo	F1, AUC, precision/recall por classe, top-3 importâncias.
Otimização	Gap vs heurística greedy, tempo de cálculo.
Operação	Missões criadas, concluídas no prazo, tempo até decisão.
Câmera	Acurácia de validação; taxa de correção da previsão.
Impacto agro	% safra salva (estimada), perda evitada em R\$, áreas atendidas.
Educacional	Usuário leigo completa a missão sem ajuda; taxa de primeiro acerto.
Segurança	Cobertura de controles, testes de acesso negado, tempo de failover do DR.
Qualidade	Tempo de resposta da API, taxa de erro, completude dos registros.

14. Escopo do protótipo mínimo

14.1 Entra no MVP

- Notebook de IA/ML com previsão de risco em agro, métricas e interpretação.
- Otimizador próprio sobre instância pequena (10–20 áreas, 2–4 recursos).
- API REST (Python) com endpoints principais e Swagger.
- Serviço .NET de Catálogo (Satélite/Sensor/Alerta) com web services e banco.
- Banco com entidades essenciais e dados sintéticos.
- Console de Missão (build web do Expo) operável até o fim.
- App de campo (Expo) com 5+ telas, AsyncStorage e câmera funcional.
- Windows Server (AD/DNS/IIS) com plano de DR espacial.
- Hands-on de segurança (recomendado: IAM/JWT) com evidências.
- Arquitetura TOGAF no Archi e doc de impacto.
- Vídeo pitch com demo ao vivo (câmera, offline e otimização no mínimo).

14.2 Não entra no MVP

- Múltiplas missões em paralelo (queimada, deslizamento) — roadmap.
- Dados privados ou contratos reais — só APIs abertas.
- Deep learning treinado do zero — ML tradicional basta.
- Hardware específico — câmera é a webcam do notebook e do celular.
- Operação real em campo — demo simulada com dado aberto e amostras controladas.

15. Plano de execução

Plano em janelas relativas, porque as datas das disciplinas precisam ser confirmadas. Marcos conhecidos: Mobile, C# e SOA entregam em 09/06/2026 (Mobile às 23h55). Recomendação: travar a versão completa pelo menos uma semana antes de qualquer entrega.

Janela	Foco	Entregas esperadas
Semana 1	Escopo e arquitetura	Doc base aprovado; nome consolidado; entidades e endpoints; divisão confirmada; arquitetura inicial no Archi.
Semana 2	Dados, modelo e serviços base	Datasets escolhidos; baseline rodando; serviço .NET de catálogo no ar; estrutura do banco.
Semana 3	Otimizador, câmera e integração	Otimização demonstrável; câmera em protótipo; web services integrados.
Semana 4	App e Console	Telas navegáveis; build web operável; integração com API; AsyncStorage.
Semana 5	Infra, segurança e docs	Windows Server + DR; hands-on de segurança; doc de compliance/Cyber; TOGAF finalizado; Swagger.
Semana 6	Estabilização e vídeo	Bugs corrigidos; demo cronometrada; vídeo gravado; arquivos congelados.

16. Divisão de trabalho

Oito matérias em cinco pessoas, com pareamentos naturais entre frentes que compartilham músculo. Atribuições de Mariana e Heloísa já confirmadas em conversa; as demais são propostas e ficam abertas a ajuste.

Pessoa	Frentes (matérias)	Responsabilidades
Mariana (confirmada)	Mobile + Design	App RN/Expo, telas, AsyncStorage, build web do Console, identidade visual, prototipação.
Heloísa (confirmada)	QA/TOGAF + Cybersecurity	Arquitetura no Archi, doc de impacto, plano de qualidade; identificação de riscos, controles e diagrama, hands-on de segurança, doc técnico ≤6p.
A confirmar (Abner)	IA & ML + Otimização	Modelo (RF/Logística), métricas, interpretação; otimizador próprio; integração com a API.
A confirmar (Beatriz)	Physical Computing (Visão)	Câmera de validação (OpenCV/MediaPipe), inferência em tempo real, integração com o app, demo ao vivo.
A confirmar (Eduardo)	Operating Systems + C# + SOA	Windows Server (AD/DNS/IIS), DR espacial; serviço .NET de Catálogo com POO/DI, web services e banco (hospedado no próprio servidor).

Observação: a frente do Eduardo concentra três matérias, mas elas são fisicamente acopladas (o serviço .NET roda no IIS do Windows Server). Se ficar pesado, o plano B do brief permite reduzir o C# a um Console simples de POO e satisfazer SOA pela própria API Python.

16.1 Responsabilidades compartilhadas

- Todos sabem explicar a tese central e a frase de posicionamento.
- Todos sabem por que recusamos museu/quiz, app de alerta e competir com a NASA.
- Todos sabem demonstrar o fluxo completo em alto nível.
- Todos participam dos ensaios do pitch e revisam o pacote final.

17. Roteiro de demonstração

30. Abertura humana: 'Espaço hoje parece coisa de gênio da NASA. Vamos provar que qualquer um pode operar uma decisão espacial real.'
31. Console de Missão abre: mapa com áreas de lavoura acendidas pelo satélite (dado real NASA/INPE).
32. Modelo indica risco em uma área e mostra o motivo principal.
33. Operador aponta a câmera em uma folha; o sistema classifica em tempo real.
34. Resultado da câmera valida a previsão e atualiza o ranking.
35. Sistema apresenta o recurso disponível (ex.: 1000 L de água, duas equipes, três dias).
36. Operador roda o otimizador; ele devolve a alocação ótima e o critério.
37. Operador aplica a primeira ação; a missão muda de estado.
38. Demo corta a conexão ao vivo: o app continua offline e enfileira a sincronização.

39. Conexão volta; a fila sincroniza; os indicadores atualizam.
40. Demo simula queda do datacenter principal; o DR espacial assume e o painel segue.
41. Convite ao jurado: 'quer rodar uma decisão?' — ele entra pela aba web e opera em poucos minutos.
42. Fechamento: ciclo Ver → Prever → Validar → Decidir → Otimizar → Agir → Medir, com a frase de posicionamento.

18. Perguntas difíceis da banca

Pergunta	Resposta recomendada
Vocês competem com a NASA na detecção?	Não. Usamos os dados deles, abertos. Nossa contribuição é decidir o que fazer com a detecção, sob recurso limitado.
Mais um projeto de NDVI?	Não. NDVI é input. O produto é o motor de decisão sob restrição, mais a democratização do uso por leigos.
A IA é só um classificador comum?	É um modelo tradicional, sim, e isso atende a disciplina. Mas com comparação de modelos, métrica, interpretação, ground-truth da câmera, e serve de insumo à otimização própria.
O otimizador é trivial?	Não. É formulado e implementado pelo time, com objetivo, restrições e instância demonstrável. Comparado a uma heurística greedy para mostrar o ganho.
O AsyncStorage é só pra cumprir tabela?	Não. O cenário rural não tem sinal contínuo. Demonstramos cortando a internet ao vivo.
Por que datacenter espacial?	Continuidade da decisão quando a infra terrestre cai. O DR replica o plano de controle e demonstramos o failover.
Por que agro?	Maior impacto da democratização para o pequeno produtor, cenário tratável e proposta original do grupo. Outras missões usam o mesmo motor (roadmap).
Onde está o 'Space Connect'?	No dado (EO real), na decisão sob recurso limitado (analogia à otimização orbital da Safe on Orbit) e na continuidade espacial (DR).
E se manipularem o dado de satélite na origem?	A câmera valida em campo (ground-truth) e fazemos cross-check entre fontes (NASA e INPE). É parte do nosso modelo de ameaças STRIDE.
Onde entra LGPD/segurança?	Dados sintéticos no MVP, PII criptografada, IAM por papel e logs. Temos um módulo de cibersegurança com riscos, controles e hands-on.
Como 8 matérias não viram 8 trabalhos colados?	Arquitetura única: cada matéria entrega um artefato que se integra via a API e o serviço .NET. C# e SOA, por exemplo, são o mesmo serviço de catálogo.
Qualquer um consegue operar mesmo?	Sim. O jurado opera uma missão completa em poucos minutos, ao vivo, sem treino.

19. Fechamento

O Orbital Academy não nasce como app educacional nem como mais um app de alerta. Nasce como uma plataforma que pega a parte da operação espacial que só especialista fazia (transformar dado em decisão sob restrição) e coloca na mão de qualquer pessoa. A Espaçoteca apoia o aprendizado; o Console de Missão e o Motor de Decisão fazem o trabalho; a câmera fecha o loop com o real; o app garante operação onde não há rede; o serviço .NET e a API integram os dados; a cibersegurança protege a integridade; e o DR espacial garante continuidade quando a infra terrestre cai.

O ciclo final é:

Ver → Prever → Validar → Decidir → Otimizar → Agir → Medir.

Esse ciclo responde diretamente ao tema Space Connect: dado espacial conectado a decisão real, com impacto mensurável, em um produto que qualquer pessoa pode operar.